

Day 4, Session 3: Vending Machine FSM - Complex Real-World Design

Concept Block (12 min)

The Vending Machine Problem

Specification:

- Item costs: **25 cents**
- Accepts coins: **5¢, 10¢, 25¢** (nickel, dime, quarter)
- When total $\geq 25¢$: Dispense item
- If total $> 25¢$: Return change
- After dispensing: Reset to accept next purchase

This FSM combines:

- State management (tracking purchase progress)
 - Arithmetic (counting money)
 - Decision logic (when to dispense/return change)
 - Multiple outputs (dispense, change amount)
-

Design Approach

Two main approaches:

Approach 1: State-Per-Amount (What we'll use)

States represent accumulated money:

- **S0**: 0¢ collected
- **S5**: 5¢ collected
- **S10**: 10¢ collected
- **S15**: 15¢ collected
- **S20**: 20¢ collected
- **S25_PLUS**: $\geq 25¢$ collected (dispense!)

Advantages:

- Clear state names (easy to understand)
- Simple state transitions
- Easy to verify

Disadvantages:

- Many states for larger amounts
 - Doesn't scale well (100¢ item = 100 states?)
-

Approach 2: Single Counter (More scalable)

One state (IDLE), with a counter variable tracking amount:

```
reg [7:0] amount_collected;  
  
if (amount_collected >= 25)  
    dispense and reset
```

Advantages:

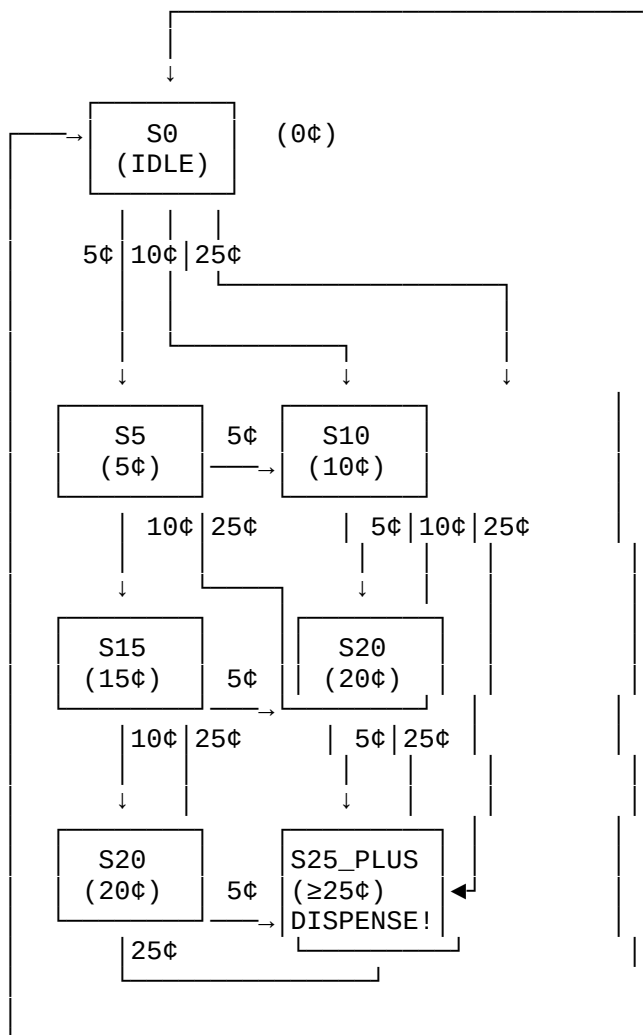
- Scales to any price
- Fewer states
- More flexible

Disadvantages:

- Mixes FSM with datapath (less clear)
- Harder to visualize

We'll use Approach 1 for learning (clearer state machine), but in industry, **Approach 2 is more common** for complex vending machines.

State Diagram: Vending Machine



All paths lead back to S0

Key features:

- From any state, any coin can be inserted
 - Once $\geq 25\text{¢}$, automatically dispense and reset
 - All successful purchases return to S0
-

Input Encoding

We have 3 coin types. How to encode?

Option 1: One-hot (3 separate inputs)

```
input wire nickel;    // 1 when 5¢ inserted
input wire dime;      // 1 when 10¢ inserted
input wire quarter;   // 1 when 25¢ inserted
```

Option 2: Binary encoded (2-bit)

```
input wire [1:0] coin;
// 00 = no coin
// 01 = nickel (5¢)
// 10 = dime (10¢)
// 11 = quarter (25¢)
```

We'll use Option 2 (binary encoded) - more compact, common in real designs.

Code & Practical Block (15 min)

Implementation: Vending Machine FSM

```
// vending_machine.v - Simple Vending Machine (25¢ item)
module vending_machine (
    input wire      clk,
    input wire      rst,
    input wire [1:0] coin,      // 00=none, 01=5¢, 10=10¢, 11=25¢
    output reg      dispense,   // 1=dispense item
    output reg [4:0] change     // Change to return (in cents)
);

    // Coin values
    localparam NO_COIN = 2'b00;
    localparam NICKEL  = 2'b01; // 5¢
    localparam DIME     = 2'b10; // 10¢
    localparam QUARTER  = 2'b11; // 25¢

    // States (amount collected)
    localparam S0       = 3'b000; // 0¢
    localparam S5       = 3'b001; // 5¢
    localparam S10      = 3'b010; // 10¢
    localparam S15      = 3'b011; // 15¢
    localparam S20      = 3'b100; // 20¢
    localparam S25_PLUS = 3'b101; //  $\geq 25\text{¢}$  (dispense)

    // State registers
```

```

reg [2:0] current_state;
reg [2:0] next_state;

// Block 1: State Register (Sequential)
always @(posedge clk or posedge rst) begin
    if (rst)
        current_state <= S0;
    else
        current_state <= next_state;
end

// Block 2: Next State Logic (Combinational)
always @(*) begin
    case (current_state)
        S0: begin // 0¢ collected
            case (coin)
                NO_COIN: next_state = S0;
                NICKEL:  next_state = S5;
                DIME:    next_state = S10;
                QUARTER: next_state = S25_PLUS; // Exact amount!
            endcase
        end

        S5: begin // 5¢ collected
            case (coin)
                NO_COIN: next_state = S5;
                NICKEL:  next_state = S10;
                DIME:    next_state = S15;
                QUARTER: next_state = S25_PLUS; // 5+25=30¢
            endcase
        end

        S10: begin // 10¢ collected
            case (coin)
                NO_COIN: next_state = S10;
                NICKEL:  next_state = S15;
                DIME:    next_state = S20;
                QUARTER: next_state = S25_PLUS; // 10+25=35¢
            endcase
        end

        S15: begin // 15¢ collected
            case (coin)
                NO_COIN: next_state = S15;
                NICKEL:  next_state = S20;
                DIME:    next_state = S25_PLUS; // 15+10=25¢ exact!
                QUARTER: next_state = S25_PLUS; // 15+25=40¢
            endcase
        end

        S20: begin // 20¢ collected
            case (coin)
                NO_COIN: next_state = S20;
                NICKEL:  next_state = S25_PLUS; // 20+5=25¢ exact!
                DIME:    next_state = S25_PLUS; // 20+10=30¢
                QUARTER: next_state = S25_PLUS; // 20+25=45¢
            endcase
        end

        S25_PLUS: begin // ≥25¢ - dispense and return to idle
            next_state = S0; // Always return to S0 after dispensing
        end

        default: next_state = S0;
    endcase
end

```

```

        endcase
    end

    // Block 3: Output Logic (Combinational - Moore)
    always @(*) begin
        // Default outputs
        dispense = 0;
        change = 5'd0;

        case (current_state)
            S0, S5, S10, S15, S20: begin
                dispense = 0; // Not enough money yet
                change = 0;
            end

            S25_PLUS: begin
                dispense = 1; // Dispense item!

                // Calculate change based on how we got here
                // We need to know previous state + coin to calculate exact
change
                // For now, we'll calculate based on total collected
                // This is a simplification - see note below
                change = 0; // Will be improved in advanced version
            end

            default: begin
                dispense = 0;
                change = 0;
            end
        endcase
    end
endmodule

```

Important Design Note: Change Calculation Challenge

Problem: To calculate exact change, we need to know:

1. Previous state (how much was collected)
2. Last coin inserted
3. Total = previous + last coin

Example:

```

S20 + DIME = S25_PLUS
Total = 20 + 10 = 30¢
Change = 30 - 25 = 5¢

```

But in Moore machine, output depends ONLY on current state!

Solutions:

Solution 1: Encode Total in State Name (What we'll do)

```

localparam S25 = 3'b101; // Exactly 25¢
localparam S30 = 3'b110; // 30¢ (return 5¢)
localparam S35 = 3'b111; // 35¢ (return 10¢)
// etc...

```

Solution 2: Use Mealy Machine

Output (change) depends on current_state AND coin input.

Solution 3: Add Amount Register (Most practical)

```
reg [5:0] total_amount; // Track exact cents collected
```

Let's implement Solution 3 (most realistic):

Improved Version: With Change Calculation

```
// vending_machine_v2.v - Vending Machine with Proper Change
module vending_machine_v2 (
    input wire      clk,
    input wire      rst,
    input wire [1:0] coin,      // 00=none, 01=5¢, 10=10¢, 11=25¢
    output reg      dispense,   // 1=dispense item
    output reg [5:0] change     // Change to return (in cents)
);

    // Coin values
    localparam NO_COIN = 2'b00;
    localparam NICKEL  = 2'b01; // 5¢
    localparam DIME     = 2'b10; // 10¢
    localparam QUARTER  = 2'b11; // 25¢

    // States
    localparam IDLE      = 1'b0; // Waiting for coins
    localparam DISPENSE  = 1'b1; // Dispensing item

    // State registers
    reg current_state;
    reg next_state;

    // Amount tracking (datapath)
    reg [5:0] amount;      // Total collected (0-63 cents)
    reg [5:0] next_amount;

    // Block 1: State Register (Sequential)
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            current_state <= IDLE;
            amount <= 6'd0;
        end
        else begin
            current_state <= next_state;
            amount <= next_amount;
        end
    end

    // Block 2: Next State and Amount Logic (Combinational)
    always @(*) begin
        // Default: stay in current state, keep amount
        next_state = current_state;
        next_amount = amount;

        case (current_state)
            IDLE: begin
                // Add coin value to amount
                case (coin)

```

```

        NO_COIN: next_amount = amount;
        NICKEL:  next_amount = amount + 6'd5;
        DIME:    next_amount = amount + 6'd10;
        QUARTER: next_amount = amount + 6'd25;
    endcase

    // Check if we have enough
    if (next_amount >= 6'd25)
        next_state = DISPENSE;
    else
        next_state = IDLE;
    end

    DISPENSE: begin
        // After dispensing, return to IDLE with amount=0
        next_state = IDLE;
        next_amount = 6'd0;
    end

    default: begin
        next_state = IDLE;
        next_amount = 6'd0;
    end
end
endcase
end

// Block 3: Output Logic (Combinational - Moore)
always @(*) begin
    case (current_state)
        IDLE: begin
            dispense = 0;
            change = 6'd0;
        end

        DISPENSE: begin
            dispense = 1;
            // Calculate change: total collected - item price
            if (amount >= 6'd25)
                change = amount - 6'd25;
            else
                change = 6'd0; // Shouldn't happen
            end
        end

        default: begin
            dispense = 0;
            change = 6'd0;
        end
    end
endcase
end

endmodule

```

Key Improvements in V2

1. Amount Tracking:

```
reg [5:0] amount; // Tracks exact total collected
```

2. Simpler State Machine:

IDLE → DISPENSE → IDLE

Only 2 states! The **amount** variable does the heavy lifting.

3. Exact Change Calculation:

```
change = amount - 6'd25;
```

4. Clean Reset:

```
amount <= 6'd0; // Reset amount after dispensing
```

Testbench: Vending Machine

```
// tb_vending_machine.v
module tb_vending_machine;

    reg        clk, rst;
    reg [1:0]  coin;
    wire       dispense;
    wire [5:0] change;

    // Coin definitions
    localparam NO_COIN = 2'b00;
    localparam NICKEL  = 2'b01;
    localparam DIME    = 2'b10;
    localparam QUARTER = 2'b11;

    vending_machine_v2 uut (
        .clk(clk),
        .rst(rst),
        .coin(coin),
        .dispense(dispense),
        .change(change)
    );

    // Clock generator
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Test stimulus
    initial begin
        $dumpfile("vending_machine.vcd");
        $dumpvars(0, tb_vending_machine);

        $display("Time | Coin      | Amount | Dispense | Change");
        $display("-----");

        // Initialize
        rst = 1; coin = NO_COIN;
        #10;
        rst = 0;
        #10;

        // Test 1: Exact change (25¢)
        $display("\n--- Test 1: Insert 25¢ (exact) ---");
        coin = QUARTER; #10;
        coin = NO_COIN; #10;

        // Test 2: Multiple coins = exact (5¢ + 10¢ + 10¢)
```

```

$display("\n--- Test 2: Insert 5¢ + 10¢ + 10¢ (exact) ---");
coin = NICKEL; #10;
coin = DIME; #10;
coin = DIME; #10;
coin = NO_COIN; #10;

// Test 3: Overpayment (10¢ + 25¢ = 35¢, change=10¢)
$display("\n--- Test 3: Insert 10¢ + 25¢ (35¢, return 10¢) ---");
coin = DIME; #10;
coin = QUARTER; #10;
coin = NO_COIN; #10;

// Test 4: Multiple small coins with overpayment
$display("\n--- Test 4: Insert 5¢ + 5¢ + 5¢ + 5¢ + 10¢ (30¢, return 5¢)
---");
coin = NICKEL; #10;
coin = NICKEL; #10;
coin = NICKEL; #10;
coin = NICKEL; #10;
coin = DIME; #10;
coin = NO_COIN; #10;

$finish;
end

// Monitor
always @(posedge clk) begin
    $display("%4t | %s | %2d¢ | %b | %2d¢",
        $time,
        (coin == NO_COIN) ? "None" :
        (coin == NICKEL) ? "5¢" :
        (coin == DIME) ? "10¢" :
        (coin == QUARTER) ? "25¢" : "???",
        uut.amount,
        dispense,
        change);
end

endmodule

```

Expected Output

Time	Coin	Amount	Dispense	Change
------	------	--------	----------	--------

 --- Test 1: Insert 25¢ (exact) ---

25	25¢	25¢	0	0¢ ← Coin inserted
35	None	0¢	1	0¢ ← DISPENSED! No change

--- Test 2: Insert 5¢ + 10¢ + 10¢ (exact) ---

55	5¢	5¢	0	0¢
65	10¢	15¢	0	0¢
75	10¢	25¢	0	0¢ ← Hit 25¢!
85	None	0¢	1	0¢ ← DISPENSED!

--- Test 3: Insert 10¢ + 25¢ (35¢, return 10¢) ---

105	10¢	10¢	0	0¢
115	25¢	35¢	0	0¢ ← Total=35¢
125	None	0¢	1	10¢ ← DISPENSED + 10¢ change! 

--- Test 4: Insert 5¢×4 + 10¢ (30¢, return 5¢) ---

145		5¢		5¢		0		0¢	
155		5¢		10¢		0		0¢	
165		5¢		15¢		0		0¢	
175		5¢		20¢		0		0¢	
185		10¢		30¢		0		0¢	← Total=30¢
195		None		0¢		1		5¢	← DISPENSED + 5¢ change! 

Compile and Test

```
iverilog -o sim/vending vending_machine_v2.v tb_vending_machine.v
vvp sim/vending
gtkwave vending_machine.vcd
```

Mini Task (10 min)

Modify the vending machine to support TWO items:

- Item A: 25¢
- Item B: 35¢

Add:

- Input: `item_select` (0=A, 1=B)
- Behavior: Dispense appropriate item when enough money collected

Hint: Change the comparison:

```
// Instead of: if (next_amount >= 6'd25)
// Use:
wire [5:0] item_price;
assign item_price = (item_select) ? 6'd35 : 6'd25;

if (next_amount >= item_price)
    next_state = DISPENSE;
```

Build and test the vending machine! Show me at least one transaction with change returned!

After this, we'll wrap up Day 4 with a **summary and FSM design best practices!** 